In The Name of God

Text Mining &

Natural Language Processing

Assignment 2

Implementing Inverted Indexing,

TF–IDF, and WordNet-Based Retrieval

Professor: Dr. Mostafa Fakhrahmad

Student: Paniz Azarnia 40435097

---

1. Problem Setting:

Ad hoc information retrieval (IR) is the classic task of finding relevant documents from a static collection in response to a dynamic, arbitrary user query. Formally, given a document collection D = {d_1, d_2, …, d_N} and a user query q, the system must return a ranked list of documents ordered by their predicted relevance to q.

Purely lexical matching algorithms (e.g., standard TF-IDF) rely on exact **term matching** between the query and the documents. While highly efficient and scalable, they suffer from three major limitations:

a. *Synonymy (Vocabulary Mismatch):* Users and authors often express the same concept using different words. For example, if a query contains the word "automobile" and a document uses "car", a lexical search engine will fail to match them, leading to false negatives (low recall).

b. *Polysemy (Lack of Word Sense Disambiguation):* A single word can have multiple meanings depending on context (e.g., "bank" as a financial institution vs. a riverbank). Lexical matching treats all occurrences of the token identically, leading to false positives (low precision).

c. *Lack of Term Relationships:* Lexical models treat terms as independent dimensions in a vector space (the Bag-of-Words assumption), completely ignoring semantic proximity or taxonomic relationships between words.

To address these limitations, modern retrieval pipelines often implement a **hybrid approach**: using an efficient lexical retriever to select a candidate pool, followed by a semantic reranking step that computes conceptual similarities.

2. Method:

The information retrieval pipeline is implemented across four modular phases:

1. Data Preprocessing (preprocess.py)

Raw documents and queries undergo a standard linguistic pipeline to ensure consistency between indexing and retrieval:

- *Tokenization:* Breaking text into individual units.
- *Lowercasing:* Converting all text to lowercase for uniformity.
- *Stopword Removal:* Eliminating high-frequency, low-utility words.
- *Stemming:* Applying NLTK's Porter Stemmer to normalize word forms.

***2**. Lexical Retrieval (index.py & ranker.py)*

- *Indexing:* Building an InvertedIndex from the preprocessed documents.
- *Scoring:* Utilizing TFIDFRanker to compute cosine similarity scores between queries and documents.
- *Candidate Selection:* Extracting the Top-K candidates for each query to serve as input for the reranking stage.

***3.** Semantic Reranking (semantic.py)*

To improve relevance beyond keyword matching, the initial Top-K results are refined using semantic analysis:

- *POS Tagging & Context Extraction:* Identifying nouns and verbs within queries and documents.
- *Semantic Similarity:* Calculating Wu-Palmer similarity scores using WordNet synsets.
- *Score Fusion:* Applying a weighted linear combination to integrate lexical and semantic scores:

$$\text{Score}=\alpha\cdot\text{Lexical}+(1-\alpha)\cdot\text{Semantic}$$

***4.** Evaluation (evaluator.py)*

The system's performance is rigorously assessed against ground truth (qrels) using the following metrics:

a. *Precision-based:* Precision@k ($k\in\{1,5,10\}$)
b. *Ranking-based:* Mean Average Precision (MAP), Mean Reciprocal Rank (MRR)
c. *Relevance-based:* NDCG@k ($k\in\{5,10\}$)

3. Preprocessing Design:

The preprocessing module is responsible for converting raw text into a normalized token sequence suitable for indexing and retrieval.

1. Text Normalization:

The TextPreprocessor class performs the following steps:

a. *Tokenization:*

Text is segmented using the regular expression pattern r'(?u)\b\w\w+\b', which extracts tokens containing at least two alphanumeric characters.

b. *Lowercasing:*

Tokens can be converted to lowercase to ensure case-insensitive matching.

c. *Stop-word Removal:*

English stop words are removed using NLTK's built-in stop-word list.

d. *Stemming:*

Porter stemming is applied to reduce words to their root forms and improve lexical matching across morphological variants.

These steps are controlled through optional parameters in the `preprocess()` method:

```
preprocess(self, text, lower_case=True, remove_stops=True, use_stemming=True)
```

2. *Data Loading*

In addition to text preprocessing, the module includes utilities for loading the dataset:

a. *Documents:* load_documents() reads JSONL documents and concatenates title and text into a single content field.

b. *Queries:* load_queries() reads query ID–text pairs from a TSV file.

c. *Relevance judgments***:** load_qrels() reads qrels from a TSV file and stores them as qrels[(query_id, doc_id)] = relevance.

3. *Vocabulary Analysis*

The function get_vocab_size() computes the number of unique tokens in a corpus after preprocessing, which is useful for analyzing vocabulary size and sparsity.

4. *Vocabulary size before and after preprocessing (Full Preprocessing):*

- Document vocab size before: 98481 Document vocab size after: 58136
- Query vocab size before: 3003 Query vocab size after: 1917

5. *One example query and one example document before and after preprocessing:*

- *Example Document :*

  Doc ID: 632589828c8b9fca2c3a59e97451fde8fa7d188d

  Original: A hybrid of genetic algorithm and particle swarm optimization for recurrent network design An evolutionary recurrent network which automates the design of recurrent neural/fuzzy networks using a new evolutionary learning algorithm is proposed in this paper. This new evolutionary learning algorithm is based on a hybrid of genetic algorithm (GA) and particle swarm optimization (PSO), and is thus called HGAPSO. In HGAPSO, individuals in a new generation are created, not only by crossover and mutation operation as in GA, but also

by PSO. The concept of elite strategy is adopted in HGAPSO, where the upper-half of the best-performing individuals in a population are regarded as elites. However, instead of being reproduced directly to the next generation, these elites are first enhanced. The group constituted by the elites is regarded as a swarm, and each elite corresponds to a particle within it. In this regard, the elites are enhanced by PSO, an operation which mimics the maturing phenomenon in nature. These enhanced elites constitute half of the population in the new generation, whereas the other half is generated by performing crossover and mutation operation on these enhanced elites. HGAPSO is applied to recurrent neural/fuzzy network design as follows. For recurrent neural network, a fully connected recurrent neural network is designed and applied to a temporal sequence production problem. For recurrent fuzzy network design, a Takagi-Sugeno-Kang-type recurrent fuzzy network is designed and applied to dynamic plant control. The performance of HGAPSO is compared to both GA and PSO in these recurrent networks design problems, demonstrating its superiority.
Fully Processed: ['hybrid', 'genet', 'algorithm', 'particl', 'swarm', 'optim', 'recurr', 'network', 'design', 'evolutionari', 'recurr', 'network', 'autom', 'design', 'recurr', 'neural', 'fuzzi', 'network', 'use', 'new', 'evolutionari', 'learn', 'algorithm', 'propos', 'paper', 'new', 'evolutionari', 'learn', 'algorithm', 'base', 'hybrid', 'genet', 'algorithm', 'ga', 'particl', 'swarm', 'optim', 'pso', 'thu', 'call', 'hgapso', 'hgapso', 'individu', 'new', 'gener', 'creat', 'crossov', 'mutat', 'oper', 'ga', 'also', 'pso', 'concept', 'elit', 'strategi', 'adopt', 'hgapso', 'upper', 'half', 'best', 'perform', 'individu', 'popul', 'regard', 'elit', 'howev', 'instead', 'reproduc', 'directli', 'next', 'gener', 'elit', 'first', 'enhanc', 'group', 'constitut', 'elit', 'regard', 'swarm', 'elit', 'correspond', 'particl', 'within', 'regard', 'elit', 'enhanc', 'pso', 'oper', 'mimic', 'matur', 'phenomenon', 'natur', 'enhanc', 'elit', 'constitut', 'half', 'popul', 'new', 'gener', 'wherea', 'half', 'gener', 'perform', 'crossov', 'mutat', 'oper', 'enhanc', 'elit', 'hgapso', 'appli', 'recurr', 'neural', 'fuzzi', 'network', 'design', 'follow', 'recurr', 'neural', 'network', 'fulli', 'connect', 'recurr', 'neural', 'network', 'design', 'appli', 'tempor', 'sequenc', 'product', 'problem', 'recurr', 'fuzzi', 'network', 'design', 'takagi', 'sugeno', 'kang', 'type', 'recurr', 'fuzzi', 'network', 'design', 'appli', 'dynam', 'plant', 'control', 'perform', 'hgapso', 'compar', 'ga', 'pso', 'recurr', 'network', 'design', 'problem', 'demonstr', 'superior']

- *Example Query:*
  Query ID: 775e829da35e9708757dbae91e99e4dd604d2204
  Original: A Systematic Study of Online Class Imbalance Learning With Concept Drift
  Fully Processed: ['systemat', 'studi', 'onlin', 'class', 'imbal', 'learn', 'concept', 'drift']

4. Indexing and Lexical Retrieval:

1. *Inverted Index:*

The system uses an **inverted index** to support efficient term-based retrieval.
Each term is mapped to a posting list containing the documents in which it appears, along with its term frequency:

term → {doc_id:term frequency}

During indexing, each document is first preprocessed using the shared preprocessing pipeline. In addition, the system keeps track of:

- the total number of documents,
- the length of each document,
- and posting lists for all terms.

This structure allows the ranker to retrieve only the documents containing query terms, rather than scanning the full collection.

2. *TF–IDF Ranking*

Lexical ranking is performed using a **TF–IDF vector space model** with cosine similarity.

3. *TF–IDF Weighting*

For each term t in a document d, the weight is computed as:

$$TF - IDF(t,d) = (1 + log(tf_{t,d})) \cdot (log(\frac{N+1}{df_t + 1}) + 1)$$

where:

- tft,d is the term frequency of t in document d,
- dft is the number of documents containing t,
- N is the total number of documents.

Both TF and IDF are smoothed using logarithmic scaling to reduce the impact of very frequent terms.

4. *Document and Query Vectors*
   - Each document is converted into a TF–IDF vector and its vector norm is precomputed.
   - Each query is processed with the same preprocessing pipeline and converted into a TF–IDF vector using the document vocabulary.
5. *Cosine Similarity*

The relevance score between a query and a document is computed using cosine similarity:

$$score(q,d) = \frac{q \cdot d}{\|q\|\|d\|}$$

Only documents that share at least one term with the query are considered as candidates. These candidates are obtained directly from the posting lists in the inverted index, which makes retrieval efficient.

6. *Ranking Procedure*
   - Sort documents by score in descending order.
   - Return the top-k ranked results.

7. *Number of unique terms:* 58136
8. *Number of postings:* 1883493
9. *3 sample posting lists (10 shown for each):*

   - 'hybrid': *[('632589828c8b9fca2c3a59e97451fde8fa7d188d', 2),*
   ('86e87db2dab958f1bd5877dc7d5b8105d6e31e46', 2),

('1d53a898850b8d055db80ba99c59c89b080dfc4c', 1),
('4d0130e95925b00a2d1ecba931a1a05a74370f3f', 1),
('383ca85aaca9f306ea7ae04fb0b6b76f1e393395', 1),
('aeb7eaf29e16c82d9c0038a10d5b12d32b26ab60', 1),
('b63a60e666c4c0335d8de4581eaaa3f71e8e0e54', 3),
('6b557c35514d4b6bd75cebdaa2151517f5e820e2', 2),
('f99a50ce62845c62d9fcdec277e0857350534cc9', 4),
('9a1b3247fc7f0abf892a40884169e0ed10d3b684', 1)]
*- **'genet':** [('632589828c8b9fca2c3a59e97451fde8fa7d188d', 2),*
('2a047d8c4c2a4825e0f0305294e7da14f8de6fd3', 2),
('ccbcaf528a222d04f40fd03b3cb89d5f78acbdc6', 1),
('c5788be735f3caadc7d0d3147aa52fd4a6036ec4', 1),
('d53432934fa78151e7b75c95093c9b0be94b4b9a', 1),
('e7317fd7bd4f31e70351ca801f41d0040558ad83', 1),
('64887b38c382e331cd2b045f7a7edf05f17586a8', 3),
('6588070d6578bc8a9d1f284f766340587501d620', 1),
('4f58366300e6031ece7b770d3cc7ecdd019ca440', 1),
('7a48f36e567b3beda7866a25e37e4a3cc668d987', 1)]
*- **'algorithm'**: [('632589828c8b9fca2c3a59e97451fde8fa7d188d', 4),*
('12f107016fd3d062dff88a00d6b0f5f81f00522d', 1),
('1ae0ac5e13134df7a0d670fc08c2b404f1e3803c', 3),
('d2018e51b772aba852e54ccc0ba7f0b7c2792115', 3),
('c9d41f115eae5e03c5ed45c663d9435cb66ec942', 2),
('f69253e97f487b9d77b72553a9115fc814e3ed51', 1),
('a72daf1fc4b1fc16d3c8a2e33f9aac6e17461d9a', 2),
('d18cc66f7f87e041dec544a0b843496085ab54e1', 1),
('4b53f660eb6cfe9180f9e609ad94df8606724a3d', 7),
('033b62167e7358c429738092109311af696e9137', 2)]

10. *Top 10 terms by document frequency (Full Preprocess):*
    -'use': 15941
    - 'paper': 11969
    - 'base': 10694
    - 'result': 10257
    - 'propos': 9010
    - 'present': 8491
    - 'system': 8349
    - 'perform': 8097
    - 'model': 7979
    - 'data': 7872

11. *Top 10 results for 3 sample dev queries, along with relevance labels (Full Preprocess):*

Query ID: 775e829da35e9708757dbae91e99e4dd604d2204
Query: A Systematic Study of Online Class Imbalance Learning With Concept Drift
Top 10 results:

1. DocID=c25a92479398cff2569ec5eab081be5fdfecca23 Score=0.3977 Relevance=0
2. DocID=789156726070dd03a8d2175157161c35c5a5b290 Score=0.3382 Relevance=0
3. DocID=3a11c1b27efaf498005197af54da14fa0960cf6b Score=0.3357 Relevance=0
4. DocID=7f75048db423526bab02c887dd36bf6d9d84b76c Score=0.2772 Relevance=1
5. DocID=73ce5b57f37858f13ec80df075b5d74164715421 Score=0.2575 Relevance=0
6. DocID=19dfe0d4ff2418f54560c6b40c4b8135f0eba7a8 Score=0.2426 Relevance=0
7. DocID=bea20c34bc3aa04c4c49361197e415b6a09e06b8 Score=0.2277 Relevance=0
8. DocID=383f12c877a5babcdd1805aae1ea9ddf64648245 Score=0.2276 Relevance=1
9. DocID=8443e3b50190f297874d2d76233f29dfb423069c Score=0.2267 Relevance=0
10. DocID=04e5b276da90c8181d6ad8397f763a181baae949 Score=0.2214 Relevance=0

==========

Query ID: e69b1314cd65a115c98082a5863b92daa4dcf9f0
Query: Automated Melanoma Recognition in Dermoscopy Images via Very Deep Residual Networks
Top 10 results:

1. DocID=6d173e78ee719a0713abdcf2110cd7f38d082ae7 Score=0.3785 Relevance=0
2. DocID=17d1949e6653ec81eb496b38f816e99572eabcb1 Score=0.3433 Relevance=0
3. DocID=920ca8ae80f659328fe6248b9ba2d40a18c4a9c4 Score=0.3266 Relevance=0
4. DocID=9ca7adb63f3259740d98f49f82fc758dcac92df6 Score=0.3184 Relevance=0
5. DocID=526f595dca4d9c61131df2dd2aa001398f6bea43 Score=0.2596 Relevance=0
6. DocID=c43dc4ae68a317b34a79636fadb3bcc4d1ccb61c Score=0.2574 Relevance=0
7. DocID=4ea19ecd2ea554dc34913b388878d864b8a7b985 Score=0.2433 Relevance=0
8. DocID=c78755a1aa972aa59c307ebc284f9e3cdea00784 Score=0.2245 Relevance=0
9. DocID=ffdceba3805493828cf9b65f29edd4d29eee9622 Score=0.2238 Relevance=0
10. DocID=8e87d95d6eb52f5245251fc30a4283f143074fbf Score=0.2235 Relevance=0

==========

Query ID: 18897a587c91e3b177de66b19fe00708ee868df8
Query: Internet of Things: a New Application for Intelligent Traffic Monitoring System
Top 10 results:

1. DocID=5978ca8e9fdf4b900b72871a0c1e6de28294dd08 Score=0.3929 Relevance=0
2. DocID=5e671ff1d980aca18cb4859f7bbe38924eb6dd86 Score=0.3576 Relevance=0
3. DocID=5e25309f3f583f6ce606128b99659dc8400b25d0 Score=0.3460 Relevance=0

4. DocID=e2d6a601e1f047322c2a524e424dd9a94a71b7ae Score=0.3376 Relevance=0
5. DocID=582e7ca855aacce883150c4bff0b0049a5c157bc Score=0.3183 Relevance=0
6. DocID=9c703258eca64936838f700b6d2a0e2c33b12b72 Score=0.3148 Relevance=0
7. DocID=a453f5cf8e3677150ec1ef2a17952492f2389df1 Score=0.3032 Relevance=0
8. DocID=f7f7082c0652ffd51e4eb362f3f6825a38c54d69 Score=0.2959 Relevance=0
9. DocID=aca8059854b85bef137ac4ded96f743a6b097d2d Score=0.2949 Relevance=0
10. DocID=1b148b263743ae21ba53c52d33ee90c18828bf83 Score=0.2934 Relevance=0

5. Semantic Method:

To complement lexical matching, the system applies a WordNet-based semantic reranking stage on the top documents retrieved by TF–IDF. The goal of this component is not to replace lexical retrieval, but to refine the ranking of the most promising candidates by considering semantic similarity between query and document terms.

1. Semantic Reranking Strategy

Given the initial top-k documents returned by the lexical ranker, the system computes an additional semantic relevance score for each candidate document and combines it with the lexical score.
The final score is computed as:

$$\text{FinalScore}(q,d)=\alpha\cdot\text{LexicalScore}(q,d)+(1-\alpha)\cdot\text{SemanticScore}(q,d)$$

where:

- LexicalScore(q,d) is the TF–IDF cosine similarity,
- SemanticScore(q,d) is the WordNet-based semantic similarity,
- α controls the balance between lexical and semantic evidence.

In the final configuration, α=0.8 was selected because it achieved the best overall MAP, while still preserving a small contribution from semantic information.

2. Context Word Extraction

To focus on semantically informative terms, the reranker first extracts **content words** from both queries and documents:

a. The text is tokenized using NLTK tokenization.
b. Tokens are POS-tagged using pos_tag.
c. Penn Treebank tags are mapped to WordNet categories.
d. Only **nouns** and **verbs** are retained.
e. These words are lemmatized using WordNetLemmatizer.

As a result, each query and document is represented as a set of normalized semantic units of the form:

(lemma,POS)

This design reduces noise from less informative words such as function words, adjectives, or punctuation, and makes WordNet lookup more reliable.

3. WordNet Similarity Computation

For each query content word, the system retrieves its WordNet synsets and compares them against the synsets of all content words in the candidate document. Semantic relatedness is computed using **Wu–Palmer similarity**, which measures similarity based on the depth of synsets in the WordNet hierarchy.

For a given query word, the system keeps the **maximum** similarity score over all document words:

$$max(w_d \in d)\, WuPalmer(w_q, w_d)$$

The document's semantic score is then computed as the **average of these best-match scores** across all

query words:

$$\text{Score}_{\text{semantic}}(Q, D) = \frac{1}{|Q_{\text{words}}|} \sum_{q \in Q_{\text{words}}} \max_{d \in D_{\text{words}}} \text{Sim}_{\text{WP}}(q, d)$$

where Q and D are the sets of extracted query and document context words.

4. Caching and Efficiency Design

Because WordNet lookup is computationally expensive, the implementation includes two caching mechanisms:

- *Context cache:* stores extracted content words for previously processed texts.
- *Synset cache:* stores WordNet synsets for previously seen (word,pos) pairs.

These caches reduce repeated tokenization, tagging, lemmatization, and synset retrieval across queries and documents.

5. How pipeline handles cases where Qwords or Dwords becomes empty after POS filtering
   - extract_context_words() keeps only alphabetic tokens tagged as nouns or verbs, then lemmatizes them.
   - If POS filtering removes everything from the query or a document, the resulting set is empty.
   - semantic_score_from_words() explicitly checks this with if not q_words or not doc_words: return 0.0.
   - So when either side is empty, the semantic module contributes no gain: sem_score = 0.0.
   - The final score then falls back to the lexical component: final_score = alpha * lexical_score + (1 - alpha) * 0.0
   - with alpha = 0.8, this becomes 0.8 * lexical_score

6. Design Trade-offs

a. *Advantages*

- *Captures semantic relatedness beyond exact keyword overlap:*

The method can identify meaningful similarity even when the query and document use different but related words.

- *Improves ranking quality modestly:*

Experimental results showed small but consistent gains over pure TF–IDF when semantic and lexical scores were combined.

- *More robust than semantic-only retrieval:*

Since lexical retrieval remains the dominant signal, the system avoids the poor performance observed when relying only on semantic similarity.

b. *Limitations*

- *High computational cost:*

The main bottleneck is repeated Wu–Palmer similarity computation across synset pairs. This makes semantic scoring much slower than lexical retrieval.

- *Restricted semantic coverage:*

Only nouns and verbs are considered. This reduces noise, but may ignore useful semantic information from adjectives or other parts of speech.

- *Candidate pool dependency:*

Because reranking is applied only to the top lexical candidates, the semantic method cannot recover relevant documents that were missed by TF–IDF in the first stage. (The semantic method is applied only to the top-K lexical candidates (e.g., top 20), rather than to the full document collection. This design was chosen for efficiency, since WordNet-based similarity is substantially slower than TF–IDF scoring.)

- *WordNet mismatch with domain-specific vocabulary:*

WordNet works best for general English vocabulary, so it may be less effective for technical, scientific, or highly specialized terms.

6. Experimental Settings:

Important hyperparameters and design choices include the preprocessing configuration (regex-based tokenization, lowercasing, stop-word removal, and Porter stemming), the TF–IDF weighting scheme with logarithmic TF and smoothed IDF, and cosine similarity for lexical ranking. Documents are represented using concatenated title and text fields, and the lexical ranker returns the top 100 candidates. Semantic reranking is then applied to the top 20 documents using WordNet-based Wu–Palmer similarity over lemmatized nouns and verbs extracted through POS tagging. Final scores are computed by linear interpolation between lexical and semantic scores, with α=0.8 selected as the best-performing setting on the development set with alpha sweeping. Efficiency is improved through precomputed document vectors, document norms, context caching, and synset caching.

Alpha: 0.0 | MAP: 0.0804 | NDCG@10: 0.1168
Alpha: 0.2 | MAP: 0.0947 | NDCG@10: 0.1386
Alpha: 0.4 | MAP: 0.1052 | NDCG@10: 0.1536

Alpha: 0.6 | MAP: 0.1098 | NDCG@10: 0.1611
Alpha: 0.8 | MAP: 0.1102 | NDCG@10: 0.1603
Alpha: 1.0 | MAP: 0.1080 | NDCG@10: 0.1579

Best Alpha Selected: 0.8 with MAP: 0.1102

7. Results:

Metric	Random	Overlap	TF-IDF (Lexical)	Final System
MAP	0.0002	0.0487	0.1080	0.1102
MRR	0.0008	0.1566	0.2858	0.2938
P@1	0.0000	0.0800	0.1771	0.1843
P@5	0.0000	0.0583	0.1183	0.1209
P@10	0.0003	0.0413	0.0817	0.0819
NDCG@10	0.0003	0.0771	0.1579	0.1603

- *Random Baseline:*

  This represents the lower bound of system performance, where documents are ranked in a completely arbitrary order. It serves as a sanity check to ensure that the system's performance is significantly better than random chance. As expected, the near-zero MAP confirms the evaluation framework's validity.
- *Overlap Baseline:*

  This baseline uses simple coordinate matching, counting the number of shared tokens between the query and the document without considering term importance (TF-IDF weights). It measures how much raw lexical overlap contributes to retrieval relevance.
- *TF-IDF Baseline (Lexical):*

  This is our primary lexical model. Unlike the Overlap baseline, it accounts for both term frequency (TF) and inverse document frequency (IDF). It effectively prioritizes documents containing rare, discriminative terms, establishing a strong baseline for relevance.
- *Final System (Semantic Reranker):*

  This system combines the TF-IDF lexical scores with a WordNet-based semantic reranker (using Wu-Palmer similarity). It aims to bridge the "vocabulary mismatch" gap by identifying conceptually similar terms that might not appear as exact matches in the lexical model.
- Performance Analysis: Which worked best?

  The Final System consistently outperformed all other baselines across every metric.

Why it worked: The lexical model (TF-IDF) provides a strong initial ranking by identifying documents with similar keywords, while the semantic reranker provides the "fine-tuning" necessary to elevate documents that are contextually or conceptually relevant even if they use different vocabulary. This successful synergy confirms that combining lexical statistics with semantic knowledge yields superior retrieval quality compared to relying on either method in isolation.

8. Ablation Study:

- *With vs. without stop-word removal,*
- *With vs. without stemming,*
- *And other conditions:*

Configuration	MAP	MRR	P@1	P@5	P@10	NDCG@5	NDCG@10
Full Preprocessing	0.1080	0.2858	0.1771	0.1183	0.0817	0.1327	0.1579
No Stemming	0.1033	0.2804	0.1814	0.1077	0.0776	0.1247	0.1509
No Stopwords Removal	0.1080	0.2873	0.1814	0.1180	0.0811	0.1330	0.1575
Lowercase Only	0.1025	0.2795	0.1786	0.1069	0.0774	0.1237	0.1502
No Preprocessing	0.0711	0.2140	0.1300	0.0797	0.0551	0.0914	0.1082

The results were obtained by performing an Ablation Study. In this study, we started with a "Full Preprocessing" pipeline and systematically disabled specific features (Stemming, Stopword Removal, Lowercasing) to observe their individual impact on the retrieval performance of the TF-IDF model.
Full Preprocessing configuration yields the highest **MAP**, **P@5**, **P@10**, and **NDCG@10**.
The high NDCG@10 score indicates that the most relevant documents are being pushed to the top of the results list, which is the most important factor for user satisfaction.

- *TF–IDF vs. TF–IDF + semantic step:*
  After running the full pipeline and saving different baseline's evaluation results in dev_metrics.json:

METRIC	TF-IDF	TF-IDF + SEMANTIC (FINAL)	ABSOLUTE CHANGE
MAP	0.1080	0.1102	+0.0022
MRR	0.2858	0.2938	+0.0079
P@1	0.1771	0.1843	+0.0071
P@5	0.1183	0.1209	+0.0026
P@10	0.0817	0.0819	+0.0001
NDCG@5	0.1327	0.1365	+0.0037
NDCG@10	0.1579	0.1603	+0.0024

The semantic step improves every reported metric, although the improvements are modest.
The stronger improvements in `P@1`, `MRR`, and `NDCG@5` indicate that semantic reranking mainly improves the **ordering of the highest-ranked documents**. It has much less effect on the overall candidate set, which is expected because it reranks documents initially retrieved by TF-IDF rather than retrieving new documents.

9. Error Analysis:

1. *Semantic hurt:*

   **Source:* query_trace.jsonl:

   - *Query Text:* Breaking the Barrier of Transactions: Mining Inter-Transaction Association Rules
   - *Query ID:* 349099074825c262b3f7c150ac8d470aabcebada
   - *Lexical top results:*
     - Rank 1: 1f9ede76dbbd6caf7e3877918fae0d421c6f180c
     - Rank 2: 191372fd366ed4e1f6f34b018731c90f617a7396
   - *Final top results after semantic rerank:*
     - Rank 1: 191372fd366ed4e1f6f34b018731c90f617a7396
     - Rank 2: 1f9ede76dbbd6caf7e3877918fae0d421c6f180c
     -

   *Source: qrels_dev.tsv:

   - *Relevance pattern from qrels:*
     - 1f9ede76dbbd6caf7e3877918fae0d421c6f180c
     - 288c67457f09c0c30cadd7439040114e9c377bc3
     - 2b0750d16db1ecf66a3c753264f207c2cb480bde
     - 3a932716ed323b247a828bd0fd8ae9b2ee0197b2
     - 780e2631adae2fb3fa43965bdeddc0f3b885e20d
   - *Root cause*

   The semantic component assigned a sufficiently high similarity score to the non-relevant document 191372fd... to move it above the relevant lexical result 1f9ede76.... .The document may contain semantically related transaction or association-rule terminology, but this broad WordNet-level similarity does not correspond to the narrower relevance criterion represented by the qrels.

---

2. *Semantic hurt:*

**Source:* query_trace.jsonl:

- *Query Text:* From Brexit to Trump: Social Media's Role in Democracy
- *Query ID:* d4dcbb547492ea63bb1dd2016f2ca1198225d78f
- *Lexical top results:*
  - Rank 1: 4b34c6fa16e48b0e5fc5b6c32514d4405382bbd3
  - Rank 2: d538f891652f0128a1017b5e13472d57f075a70c
- *Final top results after semantic rerank:*
  - Rank 1: d538f891652f0128a1017b5e13472d57f075a70c
  - Rank 2: b34c6fa16e48b0e5fc5b6c32514d4405382bbd3

*Source: qrels_dev.tsv:

- *Relevance pattern from qrels*
  - 0c46b067779d7132c5fd097baadc261348afb261 0ee93827...
  - 0ee93827fad11535a8bed25b8d00eea35f41770e
  - 42212cf0a0bd477bf8a2abb668e0189e3d5d21d2

- 5045d1e1204d754e9cae6d999f1fc03f7d990575
- 71d55130ae9a96564ec176ff081ac5cba574db5c

- *Root cause:*

Semantic reranking changed the order of the top two documents, **but neither of them is in the judged-relevant set**.

That suggests the WordNet step matched surface/semantic similarity between the retrieved candidates, but it **did not align with qrels relevance** for this query. It can be because we only reranked the top 20 documents fetched by TF-IDF for each query.

---

3. *Semantic helped:*

**Source:* query_trace.jsonl:

- *Query Text*: Record-Aware Two-Level Compression for Big Textual Data Analysis Acceleration
- *Query ID:* 1f576e4cfe7c5426e383cd06608411e81b509feb
- *Lexical top results:*
  - Rank 1: e275f643c97ca1f4c7715635bb72cf02df928d06
  - Rank 2: 6e4aa518dffbfc83f1b480b5faf4803fd915f20a
- *Final top results after semantic rerank:*
  - Rank 1: 6e4aa518dffbfc83f1b480b5faf4803fd915f20a
  - Rank 2: e275f643c97ca1f4c7715635bb72cf02df928d06

*Source: qrels_dev.tsv:

- *Relevance pattern from qrels*
  - 04e753d012b5be333e81bf856371ac77c0a46923
  - 3145fc2e5cbdf877ef07f7408dcaee5e44ba6d4f
  - 6e4aa518dffbfc83f1b480b5faf4803fd915f20a
  - 07422b9e0a868e3d09bc48c3ac98de4f4932ece7
  - 1ac7018b0935cdb5bf52b34d738b110e2ef0416a
- *Root cause*

The semantic reranker promoted a genuinely relevant document from rank 2 to rank 1. Its WordNet-based semantic score was much higher than the lexical rank-1 document, so the combined score moved it to the top. This is a clear case where semantic information improved the ranking of a relevant result.

---

4. *Semantic helped:*

**Source:* query_trace.jsonl:

- *Query Text*: Effective Inference for Generative Neural Parsing
- *Query ID:* 2cd7c3ed5a06c461b259694376820dcfcfbe94a9
- *Lexical top results:*
  - Rank 4: afdee392c887aae5fc529c815ec88cabaeba6bcf
  - Rank 5: 39f1b108687f643015f96a0c800585a44621f99c
- *Final top results after semantic rerank:*
  - Rank 4: 39f1b108687f643015f96a0c800585a44621f99c

- Rank 5: afdee392c887aae5fc529c815ec88cabaeba6bcf

*Source: qrels_dev.tsv:

- *Relevance pattern from qrels*
  - 2b920fe2d038571693bd96dafd3ed0dbadc4cb67
  - 39f1b108687f643015f96a0c800585a44621f99c
  - 715dde17239c52b3f9924a5a35edc32b0f27830b
  - 033eb044ef6a865a53878397633876827b7a8f20
  - 0874f9274a471819b5ed82a07e67003a91913b65
- *Root cause*

The semantic reranker improved the position of a document that is explicitly labeled relevant for the same query. Although the gain is smaller than in the first case, the document still moves upward from rank 5 to rank 4.

---

5. *Interesting / other:*

**Source:* query_trace.jsonl:

- *Query Text:* Indexing multi-dimensional data in a cloud system
- *Query ID:* 53115fffaa36c99a45fb7741fa74d66aa4fb8517
- *Lexical top results:*
  - Rank 1: d7fd575c7fae05e055e47d898a5d9d2766f742b9
  - Rank 3: c29920f73686404a580fa2f9bff34548fd73125a
  - Rank 4: 9c79d6f2ce0e14a2e6d9d574cdc185fa204bfae7
- *Final top results after semantic rerank:*
  - Rank 1: c29920f73686404a580fa2f9bff34548fd73125a
  - Rank 2: 9c79d6f2ce0e14a2e6d9d574cdc185fa204bfae7
  - Rank 3: d7fd575c7fae05e055e47d898a5d9d2766f742b9

*Source: qrels_dev.tsv:

- *Relevance pattern from qrels:*
  - 09d1a6f5a50a8c3e066fb05a8833bc00663ada0e72ec19e
  - 72ec19ead4007e130786bde139c56c980f5466c5
  - 0541d5338adc48276b3b8cd3a141d799e2d40150
  - 067c7857753e21e7317b556c86e30be60aa7cac0
  - 3b3367f8f0d4692abe6c111a14bb702f30ab20a3
- *Root cause*

This is an interesting failure mode: semantic reranking **strongly reordered the top results**, but the shown candidates still do not match the judged-relevant documents.

So the reranker changed the ranking signal, but it did not recover the qrels-positive documents. It can be because we only reranked the top 20 documents fetched by TF-IDF for each query.

10. Reflection:

- Systematic Weakness: *Candidate Selection Bottleneck*

  A fundamental limitation of our current pipeline is the 'Lexical Filter Bias'. By only selecting the top K (e.g., K=20) documents from the TF-IDF lexical ranker for semantic reranking, we assume that the relevant documents are already present within that initial set.

  However, because TF-IDF relies on exact term matching, it frequently fails to retrieve documents that are conceptually relevant but lexically distinct (e.g., using synonyms). Consequently, if a truly relevant document is ranked at position 21 or lower by the lexical model, the semantic reranker never sees it and has no opportunity to promote it. The semantic reranker can only optimize the *ordering* of the top 20 candidates, but it cannot fix the *recall* error introduced by the initial TF-IDF filtering. In future iterations, expanding the reranking pool size or using a more robust semantic-aware initial retriever would be necessary to mitigate this loss of potential relevance.

- *Concrete Improvement: Persistent Precomputed Embeddings and Caching*

  Currently, the system caches WordNet Synsets, but computes pairwise semantic similarities (Wu-Palmer) dynamically. A concrete improvement would be to extend this by implementing a Pairwise Semantic Cache that maps tuples of `(query_term_synset, doc_term_synset)` to their pre-calculated similarity scores. This would transition the system from O(Tree Traversal) to O(1) lookup, fundamentally eliminating the runtime bottleneck.

11. AI Usage Disclosure:

I used ChatGPT (OpenAI, GPT-5/Codex-style assistant) as a programming and writing aid for this project. It was used to help organize the system methodology, improve technical explanations, suggest CLI structure, and provide debugging/implementation guidance. All final coding decisions, experiments, and result analysis were carried out and verified by me.